

Http protocol

Hyper Text Transfer Protocol is an application protocol used to communicate and transfer data between a web browser(client) and a web server.

If you open <http://www.wikipedia.org>, your browser sends a request to the Wikipedia server, and the server sends the Wikipedia web page back to your browser.

Network Analysis using Wireshark – HTTP Protocol

In this practical, we will use Wireshark to capture and analyze HTTP packets exchanged between a web browser and a web server. This helps us understand how HTTP communication takes place on a network.

1. Make sure your computer is connected to the Internet through a Wi-Fi or Ethernet connection.
2. Open your web browser. It is recommended to clear the browser cache and cookies so that fresh HTTP packets are captured instead of cached data.
3. Open the Wireshark application.
4. Select the active network interface (Wi-Fi or Ethernet) that is currently connected to the Internet.
5. Click **Start** to begin packet capture. Wireshark will now capture all packets sent and received by your computer.
6. Open your web browser and enter the URL:
<http://gtu.ac.in>
(or any other HTTP website if the above site is unavailable).
7. Wait until the web page loads completely. During this process, your browser sends an **HTTP Request** to the web server, and the server sends an **HTTP Response** back to your browser. These packets are captured by Wireshark.
8. Return to Wireshark and click **Stop** to stop capturing packets. The captured packets will now be displayed in the packet list.
9. In the **Display Filter** bar, type:
10. `http`

and press **Enter** or click **Apply**. Only HTTP packets will now be displayed.

11. Select the first **HTTP GET** packet. In the packet details pane, observe the information such as:

- Source IP Address
- Destination IP Address
- HTTP Method (GET)
- Host
- Request URI

11. You can also filter only your computer's HTTP traffic by using the following filter:

```
http && (ip.src == <your_IP> || ip.dst == <your_IP>)
```

Replace <your_IP> with your computer's IP address. This filter displays only the HTTP packets exchanged between your computer and the web server.

For example, if your IP address is 169.1.19.87, then your filter should be `http && (ip.src == 169.1.19.87 || ip.dst == 169.1.19.87)`.

12. Observe the color coding used by Wireshark:

- **Green** – TCP packets
- **Dark Blue** – DNS packets
- **Light Blue** – UDP packets
- **Black** – TCP packets with errors or retransmissions

Result:

The HTTP Request and HTTP Response packets exchanged between the client (browser) and the web server are successfully captured and analyzed using Wireshark.

Experiment – 1

1. List up to 10 different protocols that appear in the Protocol column in the unfiltered packet-listing window.
2. What is the Internet address of the www.google.com ? What is the Internet address of your computer?
`Nslookup www.gujaratuniversity.ac.in`
3. http protocol only with Wireshark running on your computer.
4. How many packets did you capture (total of all protocols, not just HTTP)?
5. Now, use display filters to determine how many packets contain your ip address (hint: Use `ip.addr` instead of the clumsy `ip.src` or `ip.dst` format).
6. What is this filter you used? Now, reverse the filter to determine how many packets don't contain your ip address.

By inspecting and looking at the information in the HTTP GET and response messages, answer the following questions.

7. What is the IP address of your computer? Of the gtu.ac.in server?
`(nslookup gtu.ac.in)`
8. What HTTP version is your browser running? What version of HTTP is the server running?

`( HTTP browser request line (GET / HTTP/1.1)& HTTP server response line (HTTP/1.1 200 OK)`

`( http.request.method == "GET" ) (http.response)`
9. What is the status code and phrase returned from the server to your browser?(`http.response.code`)
10. What languages does your browser indicate to the server that it can accept? Which header line is used to indicate this information?
http.request or http.accept_language
- 11.

When was the HTML-file, that you have retrieved, last modified at the server? Which header line is used to indicate this information?

(When was this webpage last updated on the server)

`http.response.code == 200 && http.last_modified`

12. How many bytes of content (size of file) are returned to your browser?
Which header line is used to indicate this information?

(When the server sends the webpage (HTML file) to your browser, what is the size of that file?)

HTTP Conditional GET/Response Interaction

13. Start up the Wireshark packet sniffer, and make sure that “http” is in the display filter, so that only captured HTTP messages will be displayed in the packet-list pane.
14. Enter the following URL into your browser:
<http://gaia.cs.umass.edu/wireshark-labs/HTTP-wireshark-file1.html>
Your browser should display a very simple HTML file.
15. Quickly enter the same URL into your browser again (or simply select the refresh button on your browser).
16. Stop Wireshark packet capture.

Answer the following questions. Please explain how you can find the answers

17. Inspect the contents of the first HTTP GET request from your browser to the server. Is there an “IF-MODIFIED-SINCE” header line in the HTTP GET message? Why or why not? (1st link - no)

No. `If-Modified-Since` is not present because the browser is requesting the page for the first time. It has no previous copy of the page, so it simply asks the server to send the page.

18. Inspect the contents of the server response. Has the server explicitly returned the contents of the file? How can you tell?
19. Now inspect the contents of the second HTTP GET request from your browser to the server. Is there an “IF-MODIFIED-SINCE:” header line in the HTTP GET message? If so, what information follows the “IF-MODIFIED-SINCE:” header line? (second link - yes)

Yes. `If-Modified-Since` is present because the browser already has a copy of the page. It asks the server whether the page has changed since the last time it was downloaded. If it has not changed, the server replies with **304 Not Modified**.

If **yes**, the server sends the updated file (**200 OK**).

20. What is the HTTP status code and phrase returned from the server in response to this second HTTP GET? Has the server explicitly returned the contents of the file? Explain.

21. How many HTTP GET request messages has your browser sent? Which packet in the trace contains the request for the Bill of Rights?

`http.request.method=="GET"`

(The **packet number** of that HTTP GET packet is the answer.)

22. Which packet in the trace contains the status code and phrase associated with the response to the HTTP GET request? What is the status code and phrase in the response?

or (Which packet contains the server's response to the HTTP GET request? What is the status code and status phrase in that response?)

23. Discuss http GET and RESPONSE.

HTTP GET Request:

An **HTTP GET** request is sent by the **browser (client)** to the **server** to request a web page or other resource. It contains information such as the requested URL, HTTP version, and request headers.

HTTP Response:

An **HTTP Response** is sent by the **server** back to the **browser** after receiving the GET request. It contains the **status code** (such as **200 OK** or **304 Not Modified**), response headers, and, if required, the requested web page or file.

24. Count the total number of HTTP GET requests.

Ans.: Enter in filter: `http.request.method==GET` , then count total ----- marked.

25. How much time elapsed between the first HTTP GET request from client and the HTTP response (OK) from server?

Ans.: `http.request.method==GET` || `http.response`
(Response Time – GET Time = Elapsed Time)

(How much time did the server take to reply after the browser sent the first HTTP GET request?)

26. Identify the source and destination ports used to transfer the first HTTP GET request

27. Identify the client and server IP address in the first HTTP GET request?

Ans.: Apply filter `http.request.method==GET`, see the IP, within this you will find source IP address and destination IP address.

28. Identify the client and server MAC address in the first HTTP GET request?

Ans.: Apply filter `http.request.method==GET`, see the Ethernet, within this you will find source MAC and destination MAC.